

Security Review

Cantina Managed

SKY: STARS LZ GOVERNANCE

Cantina Managed review by:

Christoph Michel, Lead Security Researcher

M4rio.eth, Lead Security Researcher



September 4, 2026



Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
3	Findings	4
3.1	Low Risk	4
3.1.1	Configure DVNs, peers, and pinned LayerZero libraries	4
3.2	Informational	4
3.2.1	Minor issues, Typos & Documentation	4
3.2.2	LZGovBridgeReceiver public getters can swallow a payload whose selector collides	4



1 Introduction

1.1 About Cantina

Cantina is a security platform that pairs a world-class network of security researchers with agentic AI to help teams building onchain find and fix real risk. Through managed reviews, competitions, and bounties, Cantina secures protocols before vulnerabilities can be exploited.

Learn more at cantina.security

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).



2 Security Review Summary

From Aug 16th to Aug 19th the Cantina team conducted a review of [xchain-helpers](#) and [spark-gov-relay](#) on commit hashes [502a0970](#) and [49d09b0d](#) respectively. The team identified a total of **3** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	1	0	1
Gas Optimizations	0	0	0
Informational	2	0	2
Total	3	0	3

The Cantina Managed team reviewed Sky's [xchain-helpers](#) and [spark-gov-relay](#) holistically on commit hashes [4d3a848d](#) and [9bb49133](#) respectively, and concluded that all findings were addressed and no new vulnerabilities were identified.



3 Findings

3.1 Low Risk

3.1.1 Configure DVNs, peers, and pinned LayerZero libraries

Severity: Low Risk.

Context: `Deploy.s.sol#L103`.

Description: The `DeployAvalancheExecutor` → `LZReceiver` deployment uses `address(1)` for the `delegate` and `owner`, preventing configuration of the actual peer, DVNs, and pinned send/receive libraries for the L1 counterpart. It also does not configure these at all and uses LayerZero's default settings.

Recommendation: Update `DeployAvalancheExecutor` to accept and set the real peer and library addresses, configure the appropriate `UlnConfig` DVNs, and pin the send/receive libraries during deployment. Clarify if `address(1)` should really be used, effectively locking the configuration.

```
// pseudo code for the minimal expected calls
OAppCore._setPeer(EthereumEid, SparkProxy) // done already in
↳ LZReceiver.constructor()
EndpointV2.setReceiveLibrary(receiver, EthereumEid, receiveLib, 0)
EndpointV2.setConfig(receiver, receiveLib, receiveUlnConfig)
```

Sky: Acknowledged. Not in scope for this review.

Cantina Managed: Acknowledged.

3.2 Informational

3.2.1 Minor issues, Typos & Documentation

Severity: Informational.

Context: *See each case below.*

Description:

1. `Deploy.s.sol#L78`: The environment variable name does not follow the convention used by the other variables. Consider renaming the environment variable to match the existing naming convention.

Recommendation: Consider fixing the above issues.

Sky: Acknowledged. Will leave as is, especially as that specific line is not in the scope of supporting the LZ governance bridge.

Cantina Managed: Acknowledged.

3.2.2 `LZGovBridgeReceiver` public getters can swallow a payload whose selector collides

Severity: Informational.

Context: `LZGovBridgeReceiver.sol#L40`.

Description: The `GovernanceOAppReceiver` is the caller of `LZGovBridgeReceiver`. It executes:

```
(bool success, bytes memory returnData) = dstTarget.call{ value: msg.value
↳ }(dstCallData);
```

The message is encoded as:

```
[srcSender] [dstTarget] [target.fnSig] [calldata]
|-----|
```



```
this is dstCallData
```

LZGovBridgeReceiver fallback forwards message to target (LZGovBridgeReceiver.sol#L40-L47). Solidity dispatches to a named function before fallback. The contract exposes four public immutables:

```
gov0appReceiver() -> 0xddf2b4ca
srcEid()          -> 0x479cca5e
srcAuthority()    -> 0x94f9f318
target()          -> 0xd4b83992
```

If dstCallData starts with one of those selectors, the getter runs on the receiver, the call succeeds, and the payload never reaches target. The LZ nonce is still consumed.

Current payloads use queue(address[],uint256[],string[],bytes[],bool[]) = 0xd9a4cbdf, which does not collide.

A later deploy is unsafe if the calldata sent to this receiver starts with one of those four selectors.

Recommendation:

Treat those four selectors as reserved. Before wiring a new target, check the selectors it will receive against 0xddf2b4ca, 0x479cca5e, 0x94f9f318, and 0xd4b83992.

Sky: Acknowledged. Note:

- Accidental collision probability per selector is $4/2^{32} = 4/4294967296 = 9.31 \times 10^{-10} \approx 10^{-9}$.
- Any new target is expected to be covered by an integration test exercising the full bridge path, which would surface a collision immediately (the payload never reaching the target).
- Fallback + public immutables is the repo-wide convention for receivers (AMBReceiver, Optimism Receiver, ArbitrumReceiver);

Cantina Managed: Acknowledged.